Stafford's Conjecture 3.8

Challenge dossier

Christopher Albert

8 September 2026

This dossier fixes the meaning of every symbol before it is used. Section 1 defines the objects in the words of the manuscript. Section 2 quotes the conjecture and states the theorem. Section 3 defines every Lean and Mathlib construction that the challenge file uses, then lists the file. Section 4 does the same for the solution file. Section 5 lists the second, exact-source Challenge and its Solution. Section 6 is a clickable map of the imported proof architecture. Section 7 records how the pairs are checked.

1 The objects, as defined in the manuscript

Convention 1 (Field)**.** Throughout, k is a field of characteristic zero: a commutative ring in which every nonzero element is invertible and in which $1+1+\cdots+1$ is never 0. Examples are $\mathbb{Q}$, $\mathbb{R}$, $\mathbb{C}$, and every subfield of $\mathbb{C}$. No algebraic closure is assumed.

Definition 2 (Weyl algebra)**.** For $n \geq 0$, the nth Weyl algebra over k is the associative unital k-algebra

$$A_n = A_n(k) = k\langle x_1, \ldots, x_n, \partial_1, \ldots, \partial_n \rangle$$

generated by $2n$ symbols subject to the relations

$$[\partial_i, x_j] = \delta_{ij}, \qquad [x_i, x_j] = [\partial_i, \partial_j] = 0,$$

where $[a, b] = ab - ba$ and δ_{ij} is 1 for $i = j$ and 0 otherwise. It is the algebra of polynomial-coefficient differential operators on $k[x_1, \ldots, x_n]$, with ∂_i acting as $\partial/\partial x_i$. For $n = 0$ it is the field k itself. It is noncommutative for $n \geq 1$; it has no zero divisors, and its only units are the nonzero scalars.

Convention 3 (Sides and written order)**.** All ideals of Weyl algebras are right ideals, and all modules are right modules. For $a \in A_n$ the right ideal generated by a is $aA_n = \{ac : c \in A_n\}$, with the cofactor on the right. Every identity is read in the order in which its factors are written: a membership $a \in bA_n$ means $a = bc$ with c on the right. No statement here is invariant under silently transposing a product.

Definition 4 (Sum of two right ideals, and membership of 1)**.** For $d, f \in A_n$, the sum $dA_n + fdA_n$ is the right ideal $\{dR + fdS : R, S \in A_n\}$. It equals A_n if and only if it contains 1, that is, if and only if there are $R, S \in A_n$ with

$$1 = dR + fdS.$$

The element f stands to the left of d in the second summand; R and S stand to the right.

Definition 5 (Bernstein filtration and Bernstein degree)**.** The Bernstein filtration of A_n assigns degree 1 to every generator x_i and every ∂_i; its mth step is spanned by the products of at most m generators. Its associated graded ring is the commutative polynomial ring $k[x_1, \dots, x_n, \xi_1, \dots, \xi_n]$ in $2n$ variables, where ξ_i is the image of ∂_i. The Bernstein degree $\deg_B d$ of a nonzero d is the smallest m with d in the mth step. In the normal form of d as a k-linear combination of ordered monomials $x^\alpha \partial^\beta$, it is the maximum of $|\alpha| + |\beta|$ over the monomials that occur. It is 0 exactly for nonzero scalars.

Definition 6 (Linear symplectic change of generators, linear Weyl coordinate)**.** The k-span V of the $2n$ generators carries the alternating bilinear form $\omega(u, v) = [u, v] \in k$. A linear symplectic change of generators is a k-linear automorphism g of V with $\omega(gu, gv) = \omega(u, v)$; it extends uniquely to a k-algebra automorphism of A_n, because it preserves the defining relations. A linear Weyl coordinate is an element $\ell = g(x_1) \in V$ for such a g: a k-linear combination of the generators that can serve as the first coordinate of a new set of generators satisfying the Weyl relations. Equivalently, ℓ is any nonzero element of V, since every nonzero vector of a symplectic space extends to a symplectic basis.

2 The conjecture and the theorem

J. T. Stafford, *Module structure of Weyl algebras*, Journal of the London Mathematical Society (2) **18** (1978), 429–442, states on page 438:

Conjecture 3.8 (Stafford 1978, p. 438). *Let $d \in A_n$. Then there exists $f \in A_n$ such that $A_n = dA_n + fdA_n$.*

This is the complete printed statement, verified against the primary source on 2026-09-05. By the definitions above it says: there are $f, R, S \in A_n$ with $1 = dR + fdS$. The printed sentence has no hypothesis on d. For $d = 0$ the right-hand side is the zero ideal, so the statement as printed is false for $d = 0$. The paragraph that follows it in the article checks the conjecture for $n = 1$ and for monomial d, and its equivalence argument takes $d \neq 0$ explicitly. The intended statement is the one for nonzero d, and that is the statement of the theorem and of the challenge. The manuscript records this in a footnote instead of altering the quotation.

Bellamy's 2026 survey (*Module structure of Weyl algebras*, J. London Math. Soc. **113** (2026), e70373, Section 3, Conjecture 3.4) states the weaker form that every finitely generated torsion A_n-module is cyclic, notes that Stafford's formulation is stronger, and reports the problem as open.

Theorem (manuscript, Theorem 1)**.** *For every characteristic-zero field k, every $n \geq 1$, and every nonzero $d \in A_n(k)$, there exist $F, R, S \in A_n(k)$ such that*

$$1 = dR + FdS.$$

Moreover one may choose $F = \ell^{\deg_B d}$ for a linear Weyl coordinate ℓ, obtained by an invertible linear symplectic change of generators.

The first sentence is the conjecture for nonzero d. The second sentence is the exact-degree strengthening: the multiplier can be taken to be a power of a single linear Weyl coordinate, and the exponent is the Bernstein degree of d. Both sentences are proved in Lean from Mathlib. The challenge below states the first sentence; the separate exact-source Challenge in Section 5 states the strengthening. The original Challenge includes $n = 0$, where a nonzero d is a unit and one may take $R = d^{-1}$, $F = S = 0$.

3 The challenge

3.1 The Lean and Mathlib constructions used

The challenge file uses the following constructions. The descriptions of the Mathlib items are the docstrings of the pinned Mathlib revision.

`Type*` A universe-polymorphic type. Quantifying over `k : Type*` means the statement holds for a field of any size.

`Field k` Mathlib's field structure on `k`: a commutative ring with $0 \neq 1$ in which every nonzero element has an inverse.

`CharZero k` Mathlib's class: the map $\mathbb{N} \to k$, $m \mapsto 1 + \cdots + 1$ (m times), is injective. This is characteristic zero.

$\mathbb{N}$ The natural numbers $0, 1, 2, \ldots$

`Fin n` The type of natural numbers below `n`, that is $\{0, \ldots, n-1\}$. It has exactly n elements.

$\alpha \oplus \beta$ The disjoint union (sum type) of two types. Its elements are `Sum.inl a` for a in α and `Sum.inr b` for b in β.

`Matrix` ι ι **`k`** Functions $\iota \times \iota \to k$, that is square matrices indexed by ι with entries in k.

`Matrix.J l R` "The matrix defining the canonical skew-symmetric bilinear form." It is defined in `Mathlib.LinearAlgebra.SymplecticGroup` as `Matrix.fromBlocks 0 (-1) 1 0`: on the index set $l \oplus l$ the block (inl, inr) is -1, the block (inr, inl) is $+1$, and the diagonal blocks are 0.

`FreeAlgebra k X` "Given a commutative semiring `R`, and a type `X`, we construct the free unital, associative `R`-algebra on `X`. One can think of `FreeAlgebra R X` as the free non-commutative polynomial ring with coefficients in `R` and variables indexed by `X`." With X the generator set, this is $k\langle x_1, \ldots, x_n, \partial_1, \ldots, \partial_n \rangle$ before any relation is imposed.

`FreeAlgebra.`ι **`k`** "The canonical function `X → FreeAlgebra R X`": it sends an index to the corresponding generator.

`algebraMap k A` The structure map $k \to A$ of a k-algebra A, sending a scalar to the corresponding constant.

`RingQuot r` "The quotient of a ring by an arbitrary relation." For a relation `r` on a ring, `RingQuot r` is the quotient by the smallest two-sided ring congruence containing `r`. When the ring is a k-algebra, the quotient is again a k-algebra.

`def, abbrev` A definition; `abbrev` marks it as transparent to Lean's unifier, so that `WeylAlg k n` and its unfolding are interchangeable.

`Prop` The type of propositions. A term of type `P : Prop` is a proof of `P`.

`∀, ∃, →, ≠` For all, there exists, implication, and inequality.

`theorem ... := by sorry` A theorem whose proof is left open. `sorry` is the placeholder; Lean records any theorem depending on it as depending on the axiom `sorryAx`. In a challenge file it marks the hole that a solution must fill.

3.2 The file

Challenge.lean

```
import Mathlib.Algebra.RingQuot
import Mathlib.Algebra.FreeAlgebra
import Mathlib.LinearAlgebra.SymplecticGroup

/-!
# Stafford's Conjecture 3.8

J. T. Stafford, *Module structure of Weyl algebras*, J. London Math. Soc. (2)
18 (1978), 429–442, states on page 438:

> Conjecture 3.8. Let `d ∈ Aₙ`. Then there exists `f ∈ Aₙ` such that
> `Aₙ = d Aₙ + f d Aₙ`.

Here `Aₙ = Aₙ(k)` is the `n`th Weyl algebra over a field `k` of characteristic
zero, generated by `x₁, …, xₙ, ∂₁, …, ∂ₙ` with `[∂ᵢ, xⱼ] = δᵢⱼ` and all other
pairs of generators commuting. Both `d Aₙ` and `f d Aₙ` are right ideals, and
`f` stands to the left of `d`. Membership of `1` in their sum is the identity
`1 = d R + f d S` for some `R, S ∈ Aₙ`, with the factors in this order.

The printed conjecture omits the hypothesis `d ≠ 0`, without which the
conclusion fails for `d = 0`; the discussion following it in the article uses
that hypothesis. This file states the intended nonzero form, for every
characteristic-zero field and every rank `n ≥ 0`. For `n = 0` the Weyl algebra
is the field itself and a nonzero `d` is a unit.

The statement below uses only Mathlib: the Weyl algebra is the free
associative `k`-algebra on `2n` generators modulo the commutator relations
given by Mathlib's standard symplectic matrix `Matrix.J`. The compared theorem
is `universalStatement`; its `sorry` is the deliberate hole filled by
`Solution.lean`.
-/

namespace Stafford38Challenge

/-- Index type of the `2n` generators of the Weyl algebra. The left summand
`Sum.inl i` indexes the coordinate `xᵢ`, the right summand `Sum.inr i` the
momentum `∂ᵢ`. -/
abbrev PhaseVar (n : ℕ) := Fin n ⊕ Fin n

/-- The defining relations of the Weyl algebra, as a relation on the free
algebra: for every pair of generators `i, j`, the commutator
`ι i * ι j - ι j * ι i` is identified with the scalar `omega i j`. With
`omega = Matrix.J`, which is `fromBlocks 0 (-1) 1 0`, this gives
`∂ᵢ xⱼ - xⱼ ∂ᵢ = δᵢⱼ` and makes any two coordinates and any two momenta
commute. -/
def relation {k : Type*} [Field k] {n : ℕ}
    (omega : Matrix (PhaseVar n) (PhaseVar n) k)
    (a b : FreeAlgebra k (PhaseVar n)) : Prop :=
  ∃ i j,
    a = FreeAlgebra.ι k i * FreeAlgebra.ι k j -
      FreeAlgebra.ι k j * FreeAlgebra.ι k i ∧
    b = algebraMap k (FreeAlgebra k (PhaseVar n)) (omega i j)

/-- The `n`th Weyl algebra `Aₙ(k)`: the free associative unital `k`-algebra on
`PhaseVar n` modulo the relations `relation (Matrix.J (Fin n) k)`. -/
abbrev WeylAlg (k : Type*) [Field k] (n : ℕ) :=
  RingQuot (relation (k := k) (n := n) (Matrix.J (Fin n) k))
```

```
/-- Stafford's Conjecture 3.8 in its intended nonzero form: for every field
`k` of characteristic zero, every `n`, and every nonzero `d ∈ Aₙ(k)`, there are
`F, R, S ∈ Aₙ(k)` with `1 = d * R + F * d * S`. Multiplication is the algebra
product, so the written order of the factors is Stafford's. -/
def UniversalStatement : Prop :=
  ∀ (k : Type*) [Field k] [CharZero k] (n : ℕ) (d : WeylAlg k n),
    d ≠ 0 → ∃ F R S : WeylAlg k n, (1 : WeylAlg k n) = d * R + F * d * S

/-- The compared theorem. The proof is supplied by `Solution.lean`. -/
theorem universalStatement : UniversalStatement := by
  sorry

end Stafford38Challenge
```

3.3 Correspondence with Section 1

Lean	Manuscript
`PhaseVar n`	The index set of the $2n$ generators. `Sum.inl i` is x_{i+1} and `Sum.inr i` is ∂_{i+1} (indices in `Fin n` start at 0).
`relation (Matrix.J (Fin n) k)`	The Weyl relations. For $i = \operatorname{inr} a$, $j = \operatorname{inl} b$ the entry of J is δ_{ab}, giving $\partial_a x_b - x_b \partial_a = \delta_{ab}$; for $i = \operatorname{inl} a$, $j = \operatorname{inr} b$ it is $-\delta_{ab}$, the same relation with the sides exchanged; the diagonal blocks give $[x_a, x_b] = [\partial_a, \partial_b] = 0$.
`WeylAlg k n`	$A_n(k)$: the free algebra modulo those relations.
`d ≠ 0`	The nonzero hypothesis discussed in Section 2.
`1 = d * R + F * d * S`	The identity $1 = dR + FdS$, in the algebra product, with the factors in the written order.
`UniversalStatement`	The conjecture for nonzero d, over every characteristic-zero field and every $n \geq 0$.

4 The solution

4.1 The additional constructions used

`Stafford38.FoundationClosure` The module of the substantive development that exports the proved theorem `Stafford38.universalStatement : Stafford38.UniversalStatement`. Its proof uses only the axioms `propext`, `Classical.choice`, and `Quot.sound`.

`Stafford38.WeylAlg k n` The substantive development's Weyl algebra, defined in `Stafford38/Statement.lean` as `Stafford.FreeWeyl k (Fin n ⊕ Fin n) (Matrix.J (Fin n) k)`, where `Stafford.FreeWeyl k ι omega` is `RingQuot (freeWeylRelation omega)` and `freeWeylRelation` has the same defining expression as `relation` above. Hence `Stafford38.WeylAlg k n` and the challenge's `WeylAlg k n` are the same type by unfolding definitions.

`Stafford38.UniversalStatement` The same proposition as `UniversalStatement`, stated on `Stafford38.WeylAlg`.

`A ≃ₐ[k] B` A k-algebra isomorphism between A and B (Mathlib's `AlgEquiv`).

`AlgEquiv.refl` is the identity isomorphism of an algebra with itself; Lean accepts it here only because the two types are definitionally equal.

`e.injective, e.symm` An isomorphism is injective, and `e.symm` is its inverse.

`intro, rcases, refine, simpa, congrArg` Tactic steps: introduce the quantified variables; destructure the existential witnesses F, R, S and the identity from the substantive theorem; supply the witnesses of the goal; and close the remaining equality by applying `e.symm` to both sides of the known identity and simplifying, using that `e.symm` is an algebra map and so preserves 1, sums, and products in their order.

4.2 The file

Solution.lean

```
import Stafford38.FoundationClosure

namespace Stafford38Challenge

abbrev PhaseVar (n : ℕ) := Fin n ⊕ Fin n

def relation {k : Type*} [Field k] {n : ℕ}
    (omega : Matrix (PhaseVar n) (PhaseVar n) k)
    (a b : FreeAlgebra k (PhaseVar n)) : Prop :=
  ∃ i j,
    a = FreeAlgebra.ι k i * FreeAlgebra.ι k j -
      FreeAlgebra.ι k j * FreeAlgebra.ι k i ∧
    b = algebraMap k (FreeAlgebra k (PhaseVar n)) (omega i j)

abbrev WeylAlg (k : Type*) [Field k] (n : ℕ) :=
  RingQuot (relation (k := k) (n := n) (Matrix.J (Fin n) k))

def UniversalStatement : Prop :=
  ∀ (k : Type*) [Field k] [CharZero k] (n : ℕ) (d : WeylAlg k n),
    d ≠ 0 → ∃ F R S : WeylAlg k n, (1 : WeylAlg k n) = d * R + F * d * S

private def toSubstantive (k : Type*) [Field k] (n : ℕ) :
    WeylAlg k n ≃ₐ[k] Stafford38.WeylAlg k n :=
  AlgEquiv.refl

theorem universalStatement : UniversalStatement := by
  intro k _ _ n d hd
  let e := toSubstantive k n
  have hsd : e d ≠ 0 := by
    intro h
    apply hd
    apply e.injective
    simpa using h
  rcases Stafford38.universalStatement (k := k) n (e d) hsd with ⟨F, R, S, h⟩
  refine ⟨e.symm F, e.symm R, e.symm S, ?_⟩
  simpa using congrArg e.symm h

end Stafford38Challenge
```

The solution repeats the challenge definitions verbatim, so its `UniversalStatement` has exactly the challenge's type; the comparison in Section 5 checks this mechanically. The proof transports the substantive theorem along the identity isomorphism and returns the witnesses. No hypothesis is added and no factor is moved.

5 The exact-source Challenge

The original pair asks for an arbitrary multiplier F. The new pair asks for $F = \ell^{\deg_B d}$, with ℓ a linear symplectic coordinate and $n \geq 1$. The Challenge imports only Mathlib and presents the Weyl algebra exactly as in Section 3. Its degree is intrinsic to the presented element: $\deg_B d$ is the least N such that d lies in the k-span of the images of the ordered words $x^a \partial^p$ with $|a| + |p| \leq N$, where the ordered words are built recursively from the generators in the free algebra. The coordinate ℓ is the first coordinate of an invertible linear symplectic change of the generators, given by matrices M, N with $MJM^{\mathsf{T}} = J$, $NJN^{\mathsf{T}} = J$, $MN = NM = 1$. Neither a degree, nor a normal form, nor an equivalence is supplied as a hypothesis; the file below is the whole statement. The transport module `Stafford38/FixedSourceChallengeTransport.lean` repeats these definitions without importing the Challenge and proves that the intrinsic filtration is the development's Bernstein filtration and that its least level is the checked PBW normal-form degree; the two coordinate predicates are definitionally equal. The Solution combines that with the proved theorem. The consumer file `tests/FixedSourceChallengeConsumer.lean` evaluates the intrinsic degree at the unit (0, once without the transport) and at a coordinate (1), and instantiates the compared theorem at both.

FixedSourceChallenge.lean

```
import Mathlib.Algebra.RingQuot
import Mathlib.Algebra.FreeAlgebra
import Mathlib.LinearAlgebra.SymplecticGroup
import Mathlib.Order.Lattice.Nat

/-!
# Stafford's exact fixed-source strengthening

This file is the Mathlib-only challenge statement.  The Weyl algebra is the
free associative algebra on the phase variables modulo the Weyl commutator
relations.  The Bernstein filtration below is the span of the images of the
ordered PBW words of bounded total word degree, and `bernsteinDegree` is its
least membership level.  Thus the exponent in the statement is attached to
the presented element itself; it is not an extra degree or a supplied normal
form datum.
-/

namespace Stafford38FixedSourceChallenge

universe u

/-- The coordinate variables followed by the momentum variables. -/
abbrev PhaseVar (n : ℕ) := Fin n ⊕ Fin n

/-- The Weyl commutator relation on the free algebra. -/
def relation {k : Type u} [Field k] {n : ℕ}
    (omega : Matrix (PhaseVar n) (PhaseVar n) k)
    (a b : FreeAlgebra k (PhaseVar n)) : Prop :=
  ∃ i j,
    a = FreeAlgebra.ι k i * FreeAlgebra.ι k j -
      FreeAlgebra.ι k j * FreeAlgebra.ι k i ∧
    b = algebraMap k (FreeAlgebra k (PhaseVar n)) (omega i j)

/-- The presented Weyl algebra. -/
abbrev WeylAlg (k : Type u) [Field k] (n : ℕ) :=
  RingQuot (relation (k := k) (n := n) (Matrix.J (Fin n) k))

/-- A named generator in the presented quotient. -/
def generator (k : Type u) [Field k] (n : ℕ) (i : PhaseVar n) : WeylAlg k n :=
  RingQuot.mkAlgHom k (relation (k := k) (n := n) (Matrix.J (Fin n) k))
    (FreeAlgebra.ι k i)

/-- The old phase variables embedded into the next rank. -/
def oldIndex {n : ℕ} : PhaseVar n → PhaseVar (n + 1)
  | .inl i => .inl i.succ
  | .inr i => .inr i.succ
```

```
/-- Insert an old free-algebra word into the next-rank free algebra. -/
def freeOldMap (k : Type u) [Field k] (n : ℕ) :
    FreeAlgebra k (PhaseVar n) →ₐ[k] FreeAlgebra k (PhaseVar (n + 1)) :=
  FreeAlgebra.lift k (fun i => FreeAlgebra.ι k (oldIndex i))

/-- Ordered PBW words: old pairs first, then the newest coordinate and
momentum.  This is a free-algebra definition, so the filtration below is
independent of any project-specific normal-form construction. -/
def freeOrderedMonomial (k : Type u) [Field k] :
    (n : ℕ) → (Fin n → ℕ) → (Fin n → ℕ) → FreeAlgebra k (PhaseVar n)
  | 0, _, _ => 1
  | n + 1, a, p =>
      freeOldMap k n
          (freeOrderedMonomial k n (fun i => a i.succ) (fun i => p i.succ)) *
        FreeAlgebra.ι k (.inl (0 : Fin (n + 1))) ^ a 0 *
        FreeAlgebra.ι k (.inr (0 : Fin (n + 1))) ^ p 0

/-- The corresponding ordered word in the Weyl quotient. -/
def orderedMonomial (k : Type u) [Field k] (n : ℕ)
    (a p : Fin n → ℕ) : WeylAlg k n :=
  RingQuot.mkAlgHom k (relation (k := k) (n := n) (Matrix.J (Fin n) k))
    (freeOrderedMonomial k n a p)

/-- Total Bernstein weight of a phase exponent split into coordinates and
momenta. -/
def phaseDegree {n : ℕ} (a p : Fin n → ℕ) : ℕ :=
  (∑ i, a i) + ∑ i, p i

/-- The intrinsic Bernstein truncation generated by bounded ordered PBW
words. -/
def bernsteinPiece (k : Type u) [Field k] (n N : ℕ) :
    Submodule k (WeylAlg k n) :=
  Submodule.span k
    {z | ∃ a p : Fin n → ℕ,
      phaseDegree a p ≤ N ∧ z = orderedMonomial k n a p}

/-- Actual Bernstein degree: the least truncation containing the element.
The later transport proves that the defining set is nonempty and identifies
this infimum with the checked PBW normal-form degree. -/
noncomputable def bernsteinDegree (k : Type u) [Field k] {n : ℕ}
    (d : WeylAlg k n) : ℕ :=
  sInf {N : ℕ | d ∈ bernsteinPiece k n N}

/-- A linear combination of the named Weyl generators. -/
def linearCombination (k : Type u) [Field k] {n : ℕ}
    (M : Matrix (PhaseVar n) (PhaseVar n) k)
    (z : PhaseVar n → WeylAlg k n) (i : PhaseVar n) : WeylAlg k n :=
  ∑ j, algebraMap k (WeylAlg k n) (M i j) * z j

/-- The standard symplectic form in the phase-variable indexing. -/
abbrev standardForm (k : Type u) [Field k] (n : ℕ) :
    Matrix (PhaseVar n) (PhaseVar n) k := Matrix.J (Fin n) k

/-- A source coordinate obtained from an invertible symplectic linear change
of the Weyl generators. -/
def IsLinearWeylCoordinate (k : Type u) [Field k] (n : ℕ)
    (ell : WeylAlg k (n + 1)) : Prop :=
  ∃ (M N : Matrix (PhaseVar (n + 1)) (PhaseVar (n + 1)) k)
      (_hM : M * standardForm k (n + 1) * Matrix.transpose M =
        standardForm k (n + 1))
      (_hN : N * standardForm k (n + 1) * Matrix.transpose N =
        standardForm k (n + 1))
      (_hMN : M * N = 1) (_hNM : N * M = 1),
      ell = linearCombination k N (generator k (n + 1))
        (.inl (0 : Fin (n + 1)))

/-- The exact fixed-source statement, with the actual Bernstein degree of the
input operator as the source exponent. -/
def UniversalFixedSourceStatement : Prop :=
  ∀ (k : Type u) [Field k] [CharZero k] (n : ℕ)
    (d : WeylAlg k (n + 1)), d ≠ 0 →
      ∃ ell R S : WeylAlg k (n + 1),
        IsLinearWeylCoordinate k n ell ∧
          (1 : WeylAlg k (n + 1)) =
            d * R + ell ^ bernsteinDegree k d * d * S
```

```
/-- The compared theorem.  `FixedSourceSolution.lean` supplies the proof. -/
theorem universalFixedSourceStatement : UniversalFixedSourceStatement := by
  sorry

end Stafford38FixedSourceChallenge
```

FixedSourceSolution.lean

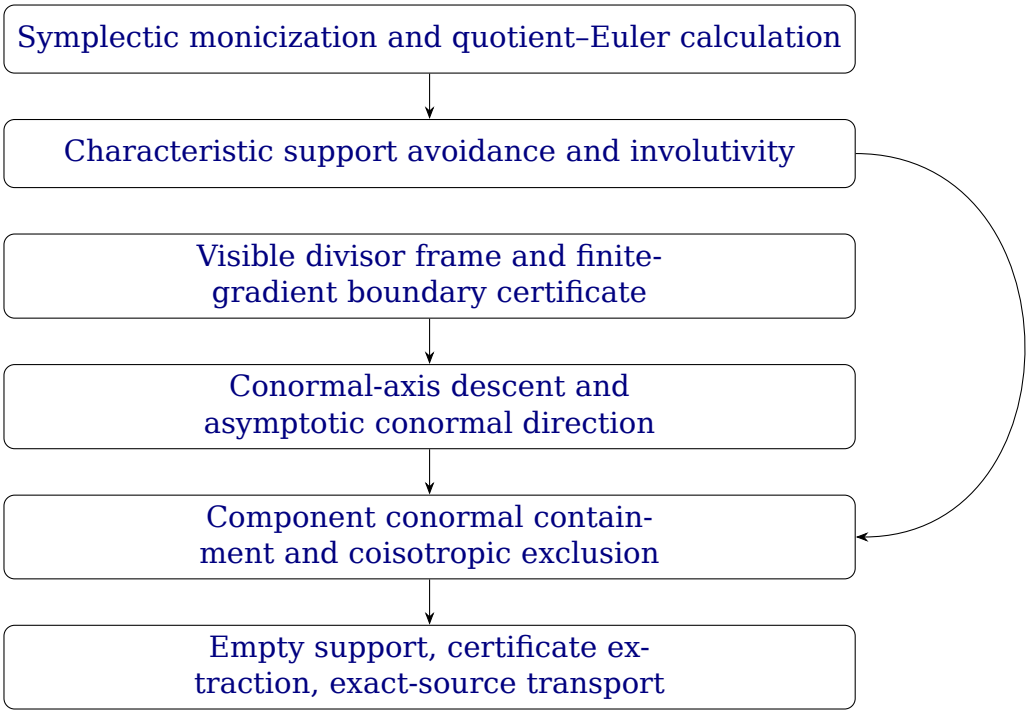
```
import Stafford38.FoundationClosure
import Stafford38.FixedSourceChallengeTransport

/-!
# Solution for the exact-source challenge

The substantive development proves the exact fixed-source theorem on the same
`RingQuot` presentation of the Weyl algebra
(`Stafford38.universalFixedSourceStatement`).  The transport module identifies
the challenge's intrinsic ordered-word filtration and its least-level degree
with the development's checked PBW normal-form filtration and degree, and the
two linear symplectic coordinate predicates are definitionally equal.  This
file only combines those facts; it adds no hypothesis and supplies no degree
or normal-form datum.
-/

namespace Stafford38FixedSourceChallenge

universe u

theorem universalFixedSourceStatement : UniversalFixedSourceStatement := by
  intro k _ _ n d hd
  obtain ⟨ell, R, S, hell, hcert⟩ :=
    Stafford38.universalFixedSourceStatement (k := k) n d hd
  have hdeg := Stafford38FixedSourceChallengeTransport.bernsteinDegree_eq k d
  refine ⟨ell, R, S,
    (Stafford38FixedSourceChallengeTransport.isLinearWeylCoordinate_iff
      k n ell).mpr hell, ?_⟩
  rw [hdeg]
  exact hcert

end Stafford38FixedSourceChallenge

#print axioms Stafford38FixedSourceChallenge.universalFixedSourceStatement
```

comparator-fixed-source.json

```
{
  "challenge_module": "FixedSourceChallenge",
  "solution_module": "FixedSourceSolution",
  "theorem_names": [
    "Stafford38FixedSourceChallenge.universalFixedSourceStatement"
  ],
  "permitted_axioms": ["propext", "Quot.sound", "Classical.choice"],
  "enable_nanoda": true
}
```

6 Architecture of the imported proof

The arrows below point from a prerequisite to the step that uses it. Each box links to its explanation. This is a summary of the terminal route in `docs/proof-graph.yaml`; the separately checked tangent-limit lemma is auxiliary and has no edge into this route.



6.1 Normalization

An invertible linear symplectic change makes the selected coordinate monic with exponent equal to the Bernstein degree. The quotient–Euler calculation retains the original operator as the right divisor in the second generator. See `Stafford38/Weyl/MonicNormalization.lean` and `Stafford38/Weyl/EulerRemainder.lean`.

6.2 Support

The canonical Koszul argument gives characteristic support avoidance. `Stafford38/Characteristic/GabberGlobalAssembly.lean` proves involutivity of the radical graded annihilator. Both feed the canonical coisotropic application in `Stafford38/Geometry/GeneralCoisotropicCanonicalAdapter.lean`.

6.3 Visible frame

In the transcendental-coordinate branch, `Stafford38/Geometry/GeneralConormalAxis.lean` obtains a normalized compatible visible divisor frame from `Stafford38/Geometry/GeneralDivisorialVisibleFrame.lean`, which is built on the retained-valuation data of `Stafford38/Geometry/ExactDivisorialVisibleFrameExistence.lean` and the generic divisorial frame construction of `Stafford38/Geometry/DivisorialVisibleFrameStageAssembly.lean`. It then applies the finite-gradient boundary certificate of `Stafford38/Geometry/FiniteGradientResidueExtension.lean`. This is the imported route; the earlier proof account cited the separately proved tangent-limit lemma here, which this route does not import.

6.4 Axis and descent

The certificate gives a conormal axis over the ground field. `Stafford38/Geometry/GeneralAsymptoticLaurentAxis.lean` combines this with the separate algebraic-coordinate

branch. Smooth-fibre vanishing and scalar-extension descent yield the projective conormal direction in `Stafford38/Geometry/GeneralAsymptoticConormal.lean`.

6.5 Exclusion

`Stafford38/Geometry/GeneralComponentConormalContainment.lean` supplies the component containment step. Together with asymptotic conormal geometry it gives the coisotropic exclusion theorem (`Stafford38/Geometry/GeneralCoisotropicExclusion.lean`, `Stafford38/Geometry/GeneralCoisotropicSets.lean`), which makes the canonical support empty.

6.6 Extraction

Certificate extraction and the inverse symplectic change give $1 = dR + \ell^{\deg_B d} dS$ in the written order. The exact-source Challenge transport proves equality of the intrinsic filtration degree and the PBW normal-form degree; it does not merely restate an arbitrary-F result.

7 How the pairs are checked

comparator.json

```
{
  "challenge_module": "Challenge",
  "solution_module": "Solution",
  "theorem_names": ["Stafford38Challenge.universalStatement"],
  "permitted_axioms": ["propext", "Quot.sound", "Classical.choice"],
  "enable_nanoda": true
}
```

Comparator exports the compared theorem (`Stafford38Challenge.universalStatement` for the original pair, `Stafford38FixedSourceChallenge.universalFixedSourceStatement` for the exact-source pair) from the challenge and solution environments separately, checks that the two statements agree, checks that the solution's proof uses only `propext`, `Quot.sound` and `Classical.choice`, and submits the exported proof to NanoDa and to Lean's own kernel. Each challenge's transitive import closure is checked to contain only Lean core and the pinned Mathlib; each solution's closure is checked not to contain either challenge.

For the original pair, both kernels accepted the exported proof in the recorded run at commit `79188b4b`, released as `v1.0.2`. The exact-source pair is new in `v1.1.0`; its run belongs to the release replay of that snapshot and is recorded in the repository's verification record only once that replay has completed. This document does not certify it.

Repository	`itpplasma/stafford38-formal`
Lean	`leanprover/lean4:v4.33.0`
Mathlib	`db584cd6d46c92f209a44c0f1c829460d327499d`
AlgebraicAnalysis `v0.3.0`, public dependency of the solutions from `v1.1.0`	`4aae47967f6ba02ffe2f639ab06564c9a9d1ecc8`
AlgebraicAnalysis `v0.2.0`, in the recorded `v1.0.2` run	`dfdd2da091a9d67e7a29cc7914f192d746a2400d`
Comparator / lean4export / NanoDa	`5756749` / `15f6055` / `68d5ca9`
Permitted axioms	`propext`, `Quot.sound`, `Classical.choice`

The verified commit and the machine-readable record of the recorded run are in the repository file `docs/verification-results.json`. That check ran in a fresh clone inside a sandbox with no access to the private research tree and no Git credentials. The `v1.1.0` snapshot is checked by the same procedure before its release.